Nombre: Landon Alexander Bautista Hernandez Karla Judith Baez Villanueva Alexa Daiana Amaya Torres	**Matrícula**: 2101662 2107930 2100425
Nombre del curso: Componentes de lógica de negocio	**Nombre del profesor**: Miguel Ángel Gómez Marroquín
Modulo Netacad: 24	**Actividad**: ACT-F03. ADR
Fecha: 16/10/2026	
Bibliografía (Formato APA) Archivo “03_Sazón_Urbano.docx” proporcionado en la plataforma TEAMS	

Architecture Decision Records

Un ADR deja por escrito una decisión de arquitectura: por qué hacía falta, qué opciones se compararon, qué se eligió y qué se acepta a cambio. Redactamos tres, cada uno con su estado, contexto, alternativas, decisión, consecuencias y trazabilidad hacia requerimientos, reglas, historias y casos de uso.

Resumen de los ADR:

- **ADR-01, patrón de lógica de negocio:** Domain Model para Pedido, Promoción, Inventario, Entrega y Pago, y Table Module solo para reportes de lectura. Se acepta más estructura inicial a cambio de que cada regla viva en un solo lugar.
- **ADR-02, Repository y Unit of Work:** un repositorio por aggregate, con interfaz en el dominio, y una unidad de trabajo por solicitud aunque toque varios aggregates. Se acepta una transacción más amplia y la ausencia de control de concurrencia.
- **ADR-03, CQRS básico:** nueve commands y cuatro queries con DTO, sobre una sola base de datos. Se acepta tener más clases a cambio de que las pantallas no dependan de los aggregates.

ADR-01, patrón de lógica de negocio

Campo	Contenido
Título	Domain Model como patrón principal; Table Module solo para consultas agregadas
Estado	Aceptado
Contexto	Sazón Urbano vende por mesa, teléfono/WhatsApp y domicilio, con 80 a 150 pedidos diarios y más de 150 ingredientes. Las reglas más delicadas dependen del estado de las cosas: cancelar según si cocina ya empezó, bloquear una promoción sin stock, dejar una transferencia pendiente hasta que alguien la verifique. Esas reglas deben cumplirse igual en cualquier canal, y el gerente quiere cambiar promociones y puntos de lealtad sin reprogramar el sistema.
Alternativas	Transaction Script: rápido de escribir, pero cada flujo repetiría las validaciones de estado y de stock. Table Module: cómodo para reportes, pero no tiene un lugar natural para el ciclo de vida del pedido ni del pago. Domain Model: exige más clases al inicio, pero deja cada regla junto al dato que protege.
Decisión	Domain Model para Pedido, Promoción, Inventario, Entrega y Pago. Table Module solo para lecturas agregadas como el cierre de caja y el reporte de ventas. Transaction Script no se usa en los cinco casos porque ninguno es una operación lineal sin reglas.
Consecuencias	Beneficios: un cambio de regla toca una sola clase y el comportamiento es igual en los tres canales. Costos: más clases y más estructura que con scripts. Riesgos: Pedido e Inventario concentran comportamiento (revisión

	SOLID, H2). Trabajo posterior: si los reportes crecen, moverlos a un modelo de lectura propio.
Trazabilidad	RF-01 a RF-05, RF-12 y RF-15 · RNF-05 · Historias 1, 5, 9, 10, 12 y 16 · CU-01 a CU-05 · RN-01 a RN-17

ADR-02, Repository y Unit of Work

Campo	Contenido
Título	Un repositorio por aggregate y una Unit of Work por solicitud
Estado	Aceptado
Contexto	Varias operaciones cambian más de un aggregate a la vez: cancelar un pedido (Pedido e Inventario), confirmar una transferencia (Pago y Cliente) y cerrar una entrega (Entrega, Pedido y Repartidor). El BRD pide modo offline con sincronización posterior (RNF-07) y el local tiene WiFi básico, así que la forma de guardar los datos puede cambiar con el tiempo.
Alternativas	SQL directo dentro de los servicios: acopla el negocio a la tecnología. Un repositorio genérico para todo: pierde el lenguaje del negocio. Un aggregate por transacción con el resto vía eventos: más puro, pero deja estados intermedios visibles. Repositorio por aggregate más Unit of Work por solicitud.
Decisión	Siete repositorios (trece métodos) con interfaz en el dominio e implementación en infraestructura, en memoria para el código base. Una Unit of Work por solicitud: se confirma al final o se deshace todo, y los eventos se publican solo después de confirmar. Sin protección contra concurrencia, para mantener el diseño simple.
Consecuencias	Beneficios: se puede cambiar de tecnología o habilitar el modo offline sin tocar el negocio, y cancelar nunca deja un pedido sin su ajuste de inventario. Costos: más interfaces y transacciones más amplias de lo que recomienda DDD estricto. Riesgos: sin control de concurrencia, dos usuarios pueden actuar sobre un estado desactualizado (por ejemplo, cancelar mientras cocina inicia); además, un Inventario único carga más de 150 ingredientes en cada operación. Trabajo posterior: agregar versionado del pedido y evaluar consistencia eventual si el negocio crece.
Trazabilidad	RNF-06 y RNF-07 · RN-04, RN-11 y RN-15 · Historias 5, 9, 12 y 16 · Hallazgos H2 y H5 de la revisión SOLID

ADR-03, CQRS básico

Campo	Contenido
Título	CQRS básico con handlers, DTO de lectura y una sola base de datos

Estado	Aceptado
Contexto	Hay operaciones que cambian el estado (registrar, cancelar, cobrar, asignar) y consultas con necesidades distintas: resumen del pedido, menú disponible, historial del cliente y cierre de caja. Cada rol ve información diferente (el mesero sus pedidos, el encargado inventario y caja, el gerente los reportes) y el BRD pide respuestas en menos de 2 segundos en operaciones críticas (RNF-01).
Alternativas	Un solo modelo para leer y escribir: expondría los aggregates a las pantallas. CQRS completo con bases separadas: complejidad que un restaurante único no justifica. CQRS básico: separación en el código sobre una sola base.
Decisión	Nueve commands y cuatro queries, cada uno con su handler. Las queries dependen de interfaces de lectura y devuelven DTO, nunca el aggregate. El cierre de caja se trata como lectura agregada. Una sola base de datos.
Consecuencias	Beneficios: responsabilidades claras, pantallas independientes del dominio y cumplimiento del principio I de SOLID. Costos: más clases, un handler por operación. Riesgos: duplicar conceptos entre el DTO y el aggregate. Trabajo posterior: separar el almacén de lectura si los reportes se vuelven pesados.
Trazabilidad	RF-02, RF-08, RF-10, RF-15 y RF-16 · RNF-01 · Historias 2, 11 y 13 · Pregunta P15 de la entrevista